

OPTIMIZACIÓN HEURÍSTICA Y APLICACIONES

TEMA 3

OPTIMIZACIÓN HEURÍSTICA BASADA EN TÉCNICAS EVOLUTIVAS

Objetivos del Tema:

- ☐ Introducir los fundamentos de los procedimientos basados en búsquedas evolutivas
- ☐ Estudiar los fundamentos de los algoritmos genéticos (GA)
- ☐ Estudiar los fundamentos de las nubes de partículas (PSO)
- ☐ Estudiar los fundamentos de la evolución diferencial (DE)

TRABAJO PRÁCTICO A ENTREGAR

La entrega correspondiente al Tema 3 consiste en la implementación de alguna de las metaheurísticas basadas en búsquedas evolutivas para resolver el problema planteado en el Tema 1. Además, en esta entrega se compararán los resultados obtenidos por la técnica utilizada en el Tema 2 con la técnica implementada en esta entrega.

La entrega se ha dividido en **dos partes**:

- **7 de Junio:** Propuesta con el planteamiento del problema (representación de las soluciones, operadores, funciones objetivo, etc.), y justificación del algoritmo seleccionado, para que los profesores de la asignatura la analicen, y comenten con los alumnos.
- **21 de Junio:** Presentación de un guión, en el que se recoja la implementación final del algoritmo, los resultados obtenidos y una discusión sobre los mismos. En este guión será necesario incluir:
 - Una descripción del funcionamiento de los **operadores** seleccionados.
 - Una explicación del **método utilizado para evaluar las soluciones** (coste + tratamiento de las restricciones).
 - Los valores a los que se han fijado los **parámetros del algoritmo** escogido.
 - Una **discusión de los resultados obtenidos**, en la que se debe poner de manifiesto:
 - si se **mejoran los puntos de partida del algoritmo** y si **se cumplen las restricciones**.
 - si el algoritmo evolutivo **mejora las soluciones obtenidas con el algoritmo de búsqueda local** utilizado en el tema anterior.

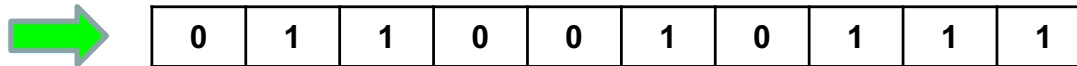
INTRODUCCIÓN

- **Algoritmos Genéticos (GA, Genetic Algorithms)**: son uno de los representantes más populares de las metaheurísticas de evolución, que se inspiraron en los **principios de herencia y selección natural de la teoría de evolución de las especies de Darwin**. A lo largo de las diferentes **iteraciones** (generaciones), mantienen una **población**/conjunto de posibles soluciones del problema (individuos de una especie), cuyos valores evolucionan hacia las mejores **soluciones** del problema (individuos mejor adaptados al medio) mediante un proceso combinado de **selección** de individuos (selección natural) y **operadores genéticos** (cruce y mutación).
- **Nubes de Partículas (PSO, Particle Swarm Optimization)**: son una metaheurística evolutiva inspirada en el **comportamiento social de las bandadas de pájaros o bancos de peces**. A lo largo de las diferentes **iteraciones**, mantienen un conjunto de posibles **soluciones** del problema (partículas) que son **desplazadas a través del espacio de búsqueda** teniendo en cuenta su **desplazamiento anterior**, el **mejor valor encontrado por la partícula a lo largo de su evolución** (el líder o **mejor local**), y el **mejor valor encontrado por todas las partículas a lo largo de la evolución** (el líder o **mejor global**).
- **Evolución Diferencial (DE, Differential Evolution)**: son una metaheurística evolutiva inspirada en el **funcionamiento de algunos métodos de optimización que modifican las soluciones de forma incremental (diferencial)**, como por ejemplo los métodos de escalada o los de gradiente. A lo largo de las diferentes **iteraciones**, las técnicas de evolución diferencial mantienen un conjunto de posibles soluciones del problema que son **desplazadas incrementalmente** teniendo en cuenta **las diferencias existentes entre parejas de soluciones** del conjunto. Por lo tanto, **los desplazamientos se adaptan a la diversidad de soluciones existentes en cada iteración**.

ALGORITMOS GENÉTICOS

Codificación de las soluciones

1. **Binaria:** las **variables de decisión del problema** se representan por medio de una cadena de longitud finita de 0s y 1s.

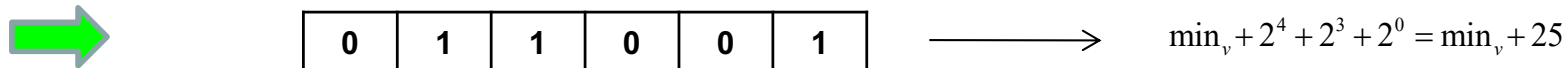


La longitud de la cadena viene determinada por el tipo de variable de decisión que se está utilizando:

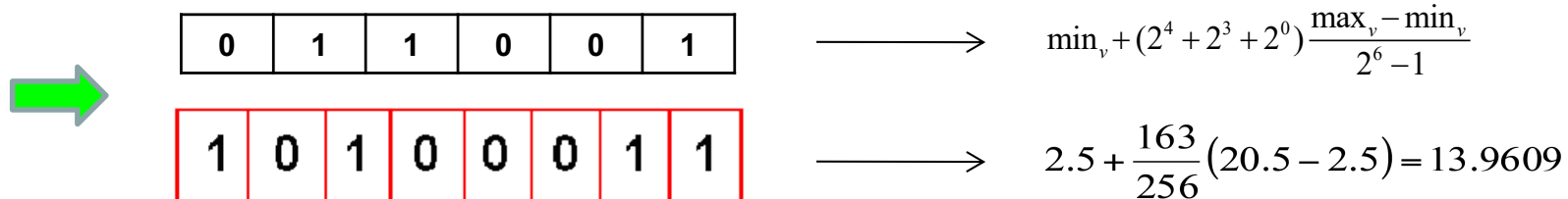
- Cuando la **variable es binaria** se puede representar mediante una cadena de longitud 1:

0

- Cuando la **variable es entera**, y sus valores máximo y mínimo se representan como $\max_v$ y el mínimo como $\min_v$, la longitud de la cadena binaria que nos permite codificar todos los valores enteros posibles es igual a $\log_2(\max_v - \min_v + 1)$.
A la hora de evaluar las funciones de mérito se convertirá por medio de un procedimiento auxiliar la cadena binaria a su correspondiente valor entero.



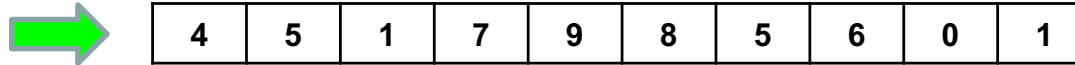
- Para representar un **variable real** con precisión infinita se necesita una cadena binaria de longitud infinita. Como este hecho no es posible, la longitud de la cadena se determina teniendo en cuenta los valores máximo y mínimo de la variable real y la precisión necesaria para representar sus valores. A la hora de evaluar las funciones de mérito se convertirá por medio de un procedimiento auxiliar la cadena binaria a su correspondiente valor real.



ALGORITMOS GENÉTICOS

Codificación de las soluciones

2. **Entera:** las variables se representan con cadenas de números enteros.

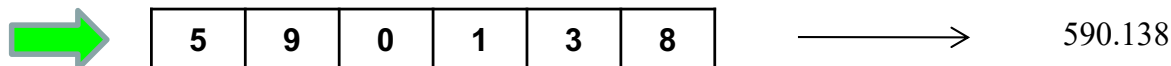


La longitud de la cadena depende del tipo de variable de decisión que se está utilizando:

- Cuando la **variable es entera** se puede representar mediante una cadena de longitud 1:

27

- Cuando la **variable es real**, la longitud de la cadena se determina teniendo en cuenta los valores máximo y mínimo de la variable real y la precisión necesaria para representar sus valores. A la hora de evaluar las funciones de mérito se convertirá, por medio de un procedimiento auxiliar, la cadena entera a su correspondiente valor real. El siguiente esquema muestra una transformación posible en la que se considera que la cadena representa un numero en base 10 con tres cifras decimales (asociada a los tres últimos dígitos).



3. **Real:** las variables se representan directamente con los tipos de variables que los lenguajes de programación proporcionan directamente para representar números reales (float, double). La precisión, en este caso, viene determinada para la precisión propia del tipo de variable elegida. De hecho, es conveniente tener en cuenta que los tipos de variables reales existentes en los lenguajes de programación codifican los valores reales utilizando unas codificaciones binarias determinadas. La diferencia principal entre el uso de codificación binaria o real en los GAs se encuentra en las operaciones realizadas por los operadores de cruce y mutación.

ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate ()

2 **repeat**:

3 **parents**=Select (**x**)

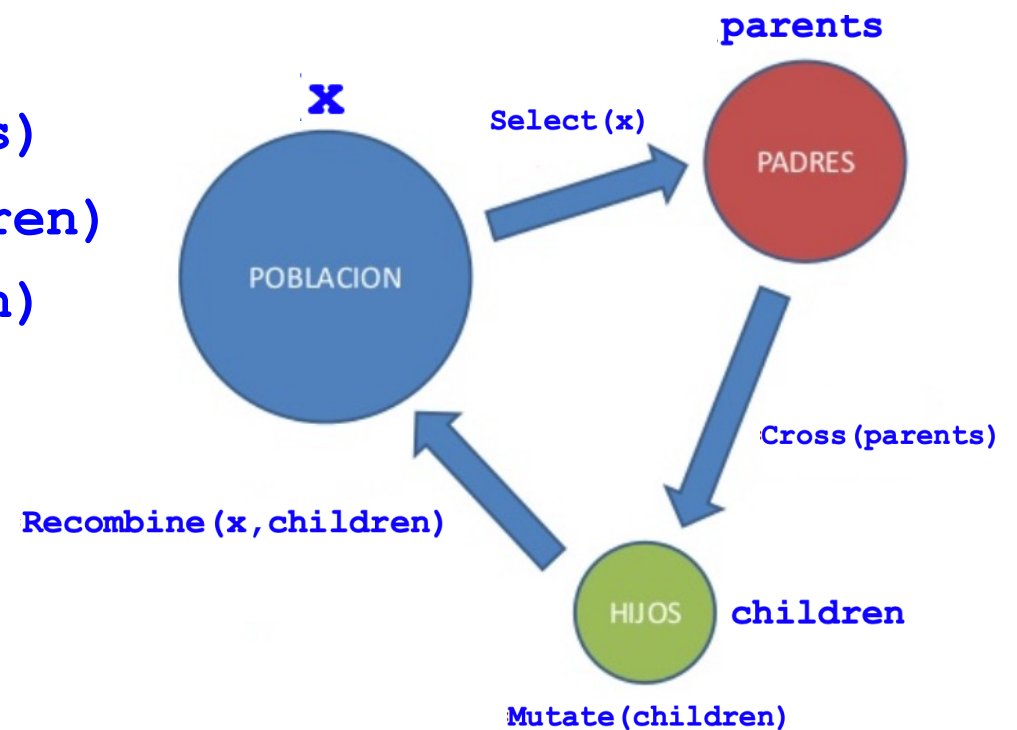
4 **children**=Cross (**parents**)

5 **children**=Mutate (**children**)

6 **x**=Recombine (**x**, **children**)

7 **until** Stop (**x**)

8 **return** **x**



ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate () ←

2 **repeat**:

3 **parents**=Select (**x**)

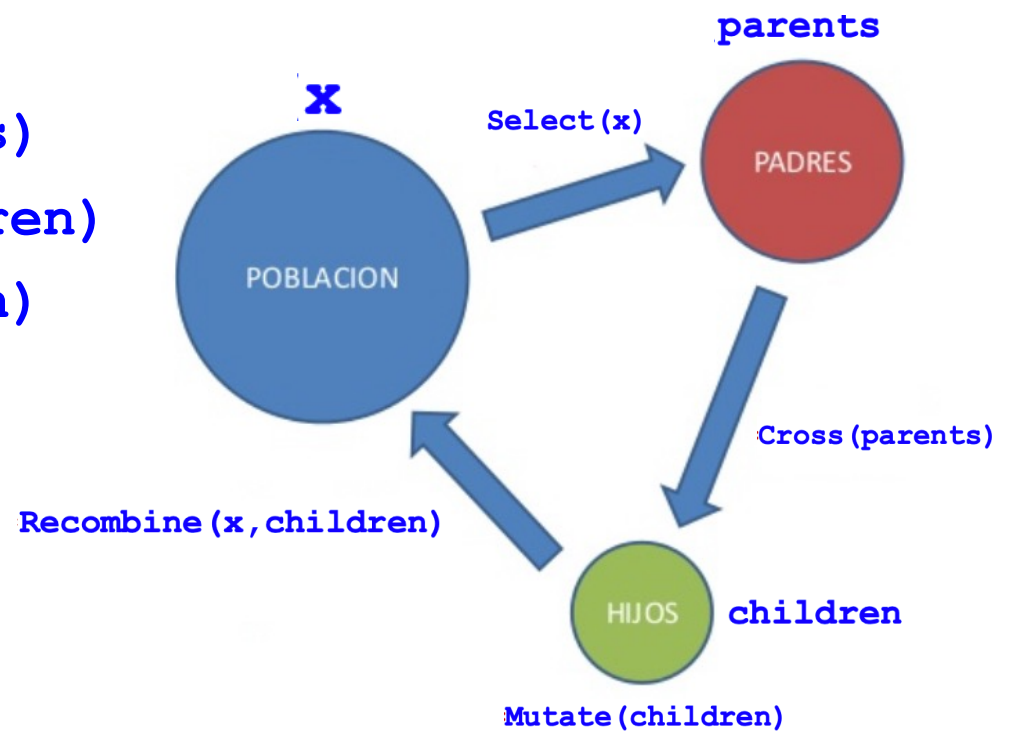
4 **children**=Cross (**parents**)

5 **children**=Mutate (**children**)

6 **x**=Recombine (**x**, **children**)

7 **until** Stop (**x**)

8 **return** **x**



ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate ()

2 **repeat**:

3 **parents**=Select (**x**)

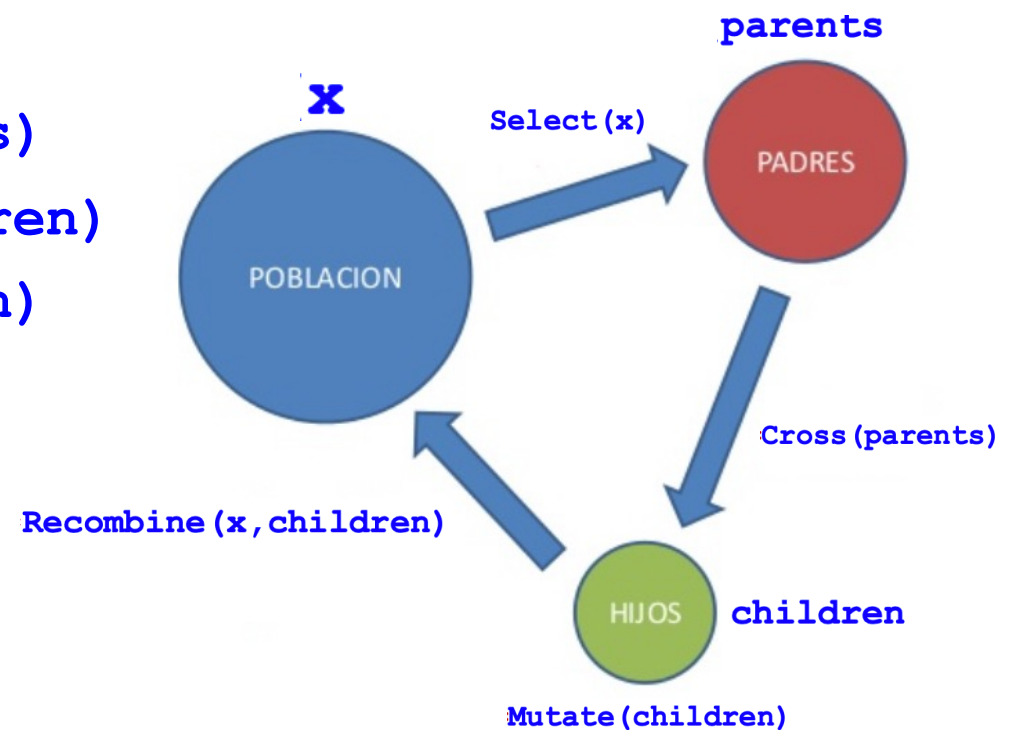
4 **children**=Cross (**parents**)

5 **children**=Mutate (**children**)

6 **x**=Recombine (**x**, **children**)

7 **until** Stop (**x**) ←

8 **return** **x**



ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate ()

2 **repeat**:

3 **parents**=Select (**x**) 

4 **children**=Cross (parents)

5 **children**=Mutate (children)

6 **x**=Recombine (**x**, children)

7 **until** Stop (**x**)

8 **return** **x**

ALGORITMOS GENÉTICOS

Función 1: Select genérica

```
parents=Select (x,Np)
```

```
1 for i=1:N
```

```
2     val(i)=Obj (x (i)) ←
```

```
3 end
```

```
4 i=1
```

```
5 while i < Np
```

```
6     parents(i,1)=SelectBasic(x,val) ←
```

```
7     parents(i,2)=SelectBasic(x,val)
```

```
8     if valid(parents(i,:)) ←
```

```
9         i=i+1;
```

```
10    end
```

```
11end
```

```
12return parents
```

ALGORITMOS GENÉTICOS

FUNCIÓN DE SELECCIÓN – Métodos de torneo

En los métodos de torneo se eligen los padres en dos etapas sucesivas:



1. Se selecciona aleatoriamente, sin tener en cuenta el valor de las funciones de mérito, un subconjunto de tamaño T de las soluciones de la población actual x .
2. Se elige uno de los individuos del subconjunto teniendo en cuenta los valores de sus funciones de mérito.

Según la forma de proceder de la segunda etapa, se distinguen dos métodos de torneo:

- Torneo determinístico: Entre todos los individuos del subconjunto elegido en la primera fase, en la segunda se elige el mejor (de acuerdo con la definición de óptimo utilizada en el algoritmo).

Ejemplo: considerando que estamos en un problema de maximización

Individuo	1	2	3	4	5	6	7	8	9	10
Función Merito	8	2	17	7	2	12	11	7	3	7



Se eligen, con $T=4$:
(2, 3, 4, 8) $\rightarrow$ 3
(1, 5, 6, 9) $\rightarrow$ 6

ALGORITMOS GENÉTICOS

FUNCIÓN DE SELECCIÓN – Métodos de torneo

- **Torneo Probabilístico:** Utiliza una etapa adicional intermedia para decidir si se elige el individuo mejor o el peor, a partir de un numero r generado aleatoriamente siguiendo una distribución uniforme entre 0 y 1, y de una probabilidad $p > 0.5$ que determina cuando se elige el mejor.

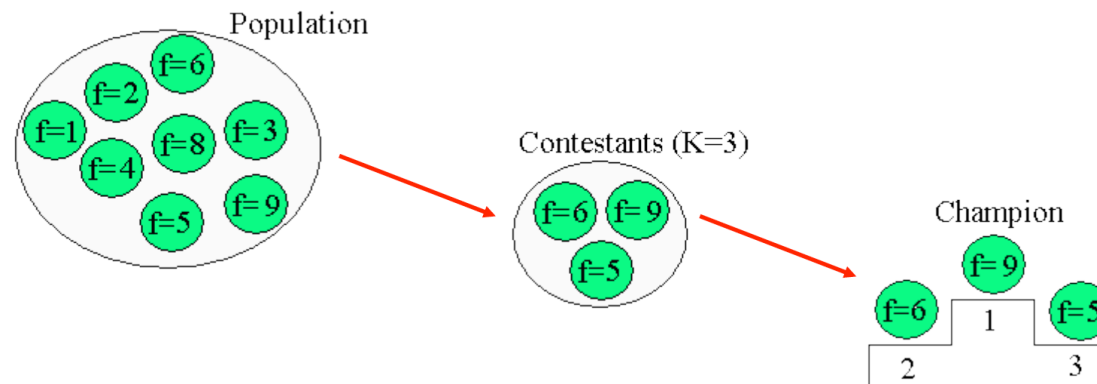
Ejemplo: considerando que estamos en un problema de maximización y que $p = 0.7$



Individuo	1	2	3	4	5	6	7	8	9	10
Función Merito	8	2	17	7	2	12	11	7	3	7



Se eligen, con $T=4$: (2, 3, 4, 8) & $r=0.5 \rightarrow 3$
(1, 5, 6, 9) & $r=0.7 \rightarrow 5$



ALGORITMOS GENÉTICOS

FUNCIÓN DE SELECCIÓN – Métodos proporcionales

Una vez **asignada la aptitud A_i a cada individuo**, se puede proceder a seleccionar aquellos que tienen una mejor aptitud utilizando diferentes procedimientos:

- **Método de la ruleta**: es uno de los métodos más simples y populares, implementable a través de las siguientes etapas:

1. **Calcular la suma de la aptitud de todos los individuos: $S = \sum A_i$.**
2. **Generar un número aleatorio r siguiendo la distribución uniforme entre 0 y S .**
3. **Recorrer los individuos de la población, sumando su aptitud, hasta que la suma sea igual o mayor a r .**
4. **Elegir como individuo aquel que hace que la suma exceda el valor de r .**



Esta forma de proceder requiere que la aptitud de todos los individuos sea positiva, y constituye un **método preservativo**, en el que **el peor individuo puede ser elegido varias veces, aunque la probabilidad de este hecho sea pequeña**.

Ejemplo:

Individuo	1	2	3	4	5	6	7	8	9	10
Aptitud	8	2	17	7	2	12	11	7	3	7
Total	8	10	27	34	36	48	59	66	69	76

No. aleatorio		23	49	76	13	1	27	57		
Individuo elegido		3	7	10	3	1	3	7		

ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate ()

2 **repeat**:

3 **parents**=Select (**x**)

4 **children**=Cross (parents)



5 **children**=Mutate (children)

6 **x**=Recombine (**x**, children)

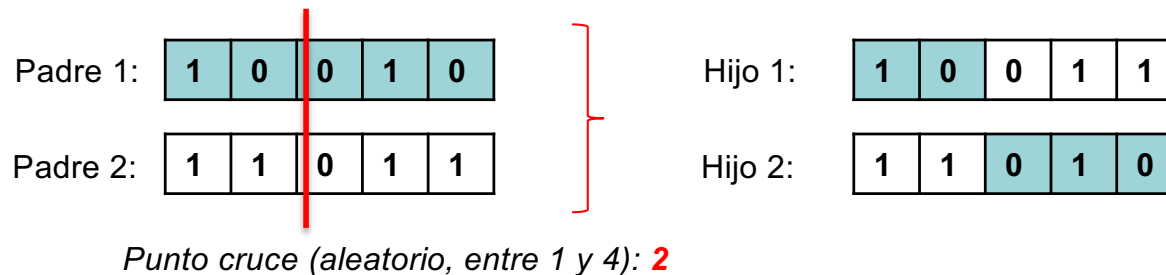
7 **until** Stop (**x**)

8 **return** **x**

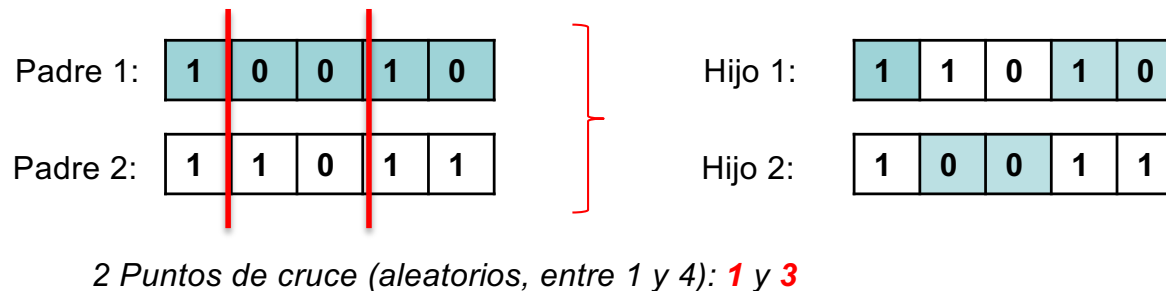
ALGORITMOS GENÉTICOS

OPERADOR DE CRUCE – Cruces por puntos

- **Cruce por 1 punto:** se determina de forma aleatoria, siguiendo una distribución habitualmente uniforme, el punto de cruce de las dos cadenas, y los hijos se crean concatenando la cadena de un padre con la del otro.

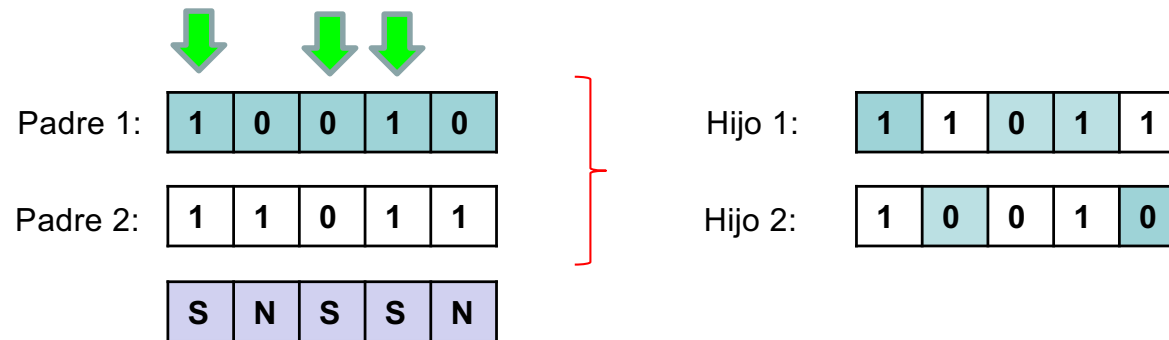


- **Cruce por N puntos:** se determinan de forma aleatoria, siguiendo una distribución habitualmente uniforme, los puntos de cruce de las dos cadenas, y los hijos se crean concatenando, de forma alternada, las cadenas de un padre con las del otro.



ALGORITMOS GENÉTICOS

- **Cruce uniforme:** se determina de forma aleatoria, a partir de una probabilidad p , si cada elemento de la cadena del primer padre pertenece al primer hijo o no.

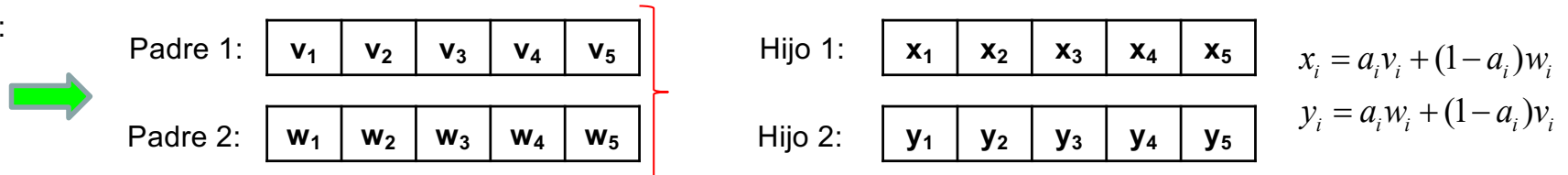


Aleatoriamente se determina que los elementos 1, 3 y 4 se toman del mismo padre y los elementos 2 y 5 del contrario

ALGORITMOS GENÉTICOS

OPERADOR DE CRUCE – Cruces para representaciones reales

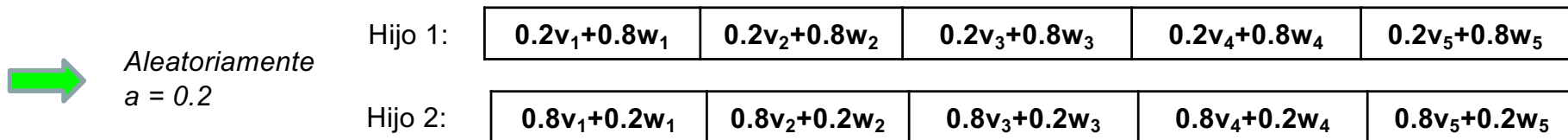
- Cruces aritméticos:** esta denominación engloba a un conjunto de cruces fundamentados en la idea de realizar la siguiente **combinación lineal de los valores de los padres, en la que el parámetro a_i toma diferentes valores**, según sea el tipo de cruce:



- Cruce aritmético total:** En este caso, todos los valores de la cadena son combinados utilizando **el mismo valor de a , que se elige aleatoriamente según una distribución uniforme entre 0 y 1**, para todos los elementos de la cadena.

$$a \sim U(0,1)$$

$$a_i = a$$

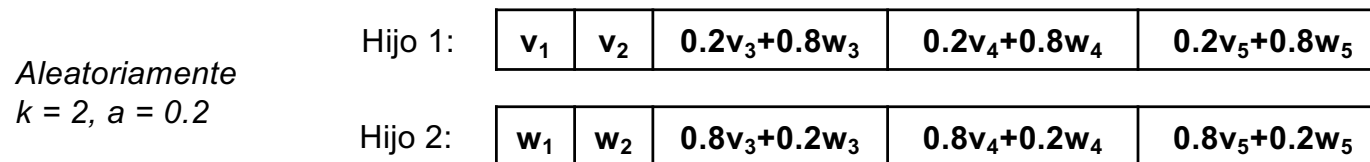


- Cruce intermedio:** en este caso, **se elige aleatoriamente un punto de cruce k a partir del cual se combinan los valores de los padres, y un valor fijo a entre 0 y 1 para todos los valores que se combinan.**

$$k \sim U(\text{PuntosCruce})$$

$$a \sim U(0,1)$$

$$a_i = \begin{cases} 1 & i \leq k \\ a & \text{c.c.} \end{cases}$$



ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate ()

2 **repeat**:

3 **parents**=Select (**x**)

4 **children**=Cross (parents)

5 **children**=Mutate (children) 

6 **x**=Recombine (**x**, children)

7 **until** Stop (**x**)

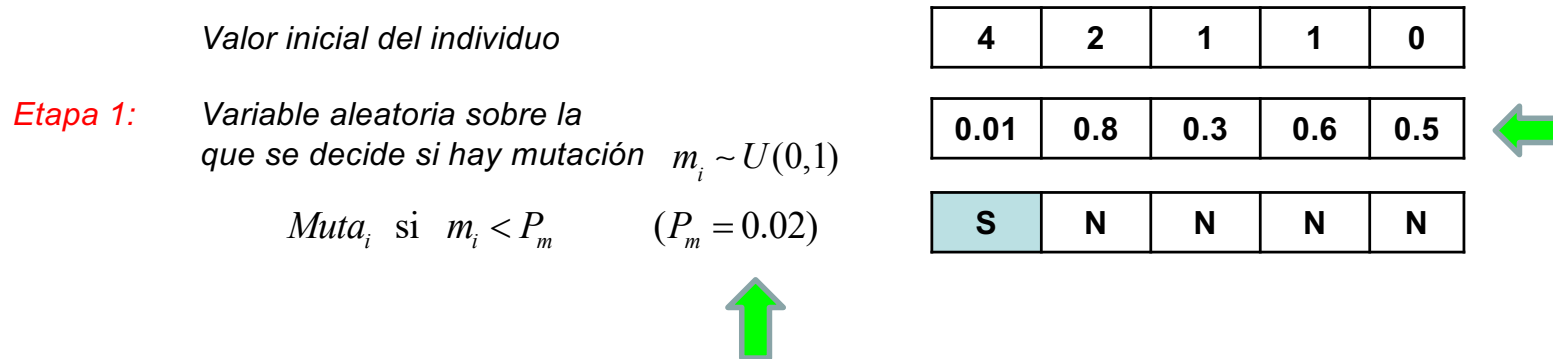
8 **return** **x**

ALGORITMOS GENÉTICOS

OPERADOR DE MUTACIÓN

Para las **codificaciones binarias, enteras y reales**, podemos distinguir dos etapas en su funcionamiento:

1. Se calculan las posiciones de las cadenas cuyos valores van a ser mutados, habitualmente utilizando una función aleatoria que **determina, a través de una probabilidad de mutación P_m , si cada punto de la cadena tiene que ser mutado o no.**



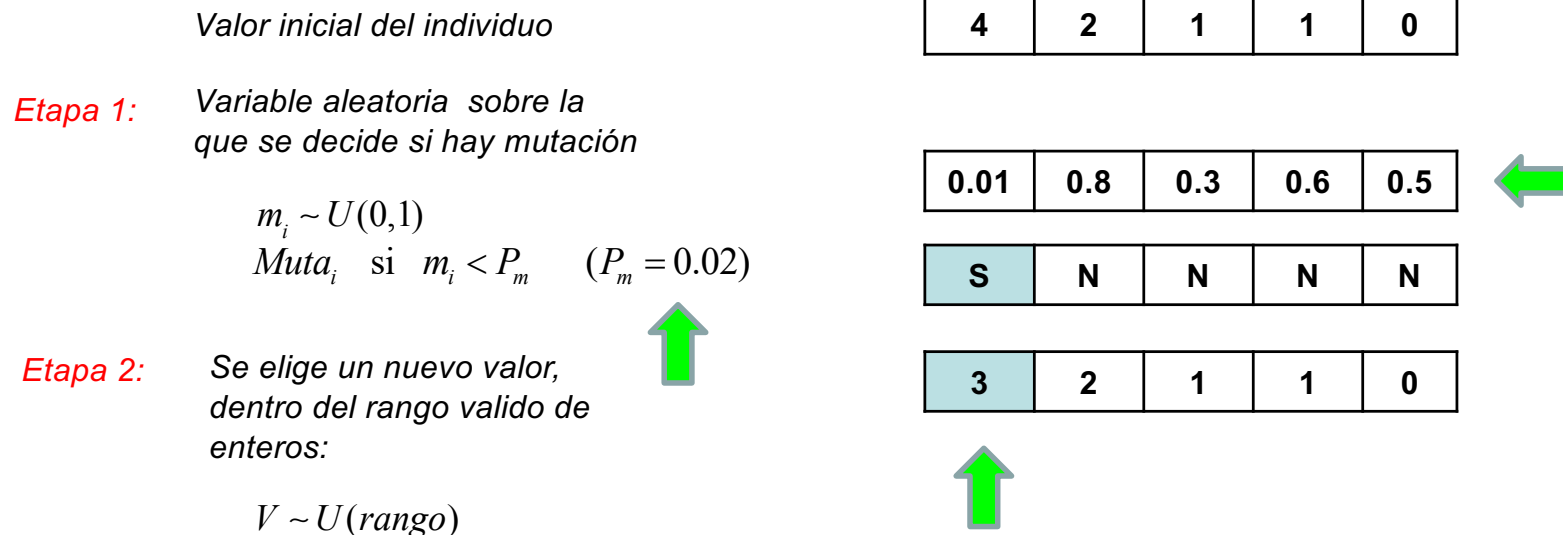
Para evitar que se realicen muchas mutaciones, es conveniente elegir valores bajos de la probabilidad de mutación.

Mientras algunos autores recomendaban inicialmente valores fijos e independientes de la longitud de la cadena (un P_m **perteneciente a $[0.001, 0.01]$**), en la actualidad se considera un valor apropiado el inverso de la longitud de la cadena. Por otra parte, algunos GA utilizan probabilidades de mutación elevadas al principio, y poco a poco, las van reduciendo.

ALGORITMOS GENÉTICOS

2. Se calculan los nuevos valores asociados a la posiciones de la cadena que deben mutar:

- En el **caso binario**, se elige el valor contrario al que tienen.
- En el **caso entero**, cualquier valor del rango valido de los valores enteros:



- En el **caso real**, se eligen los valores siguiendo alguna de las estrategias que veremos a continuación:

ALGORITMOS GENÉTICOS

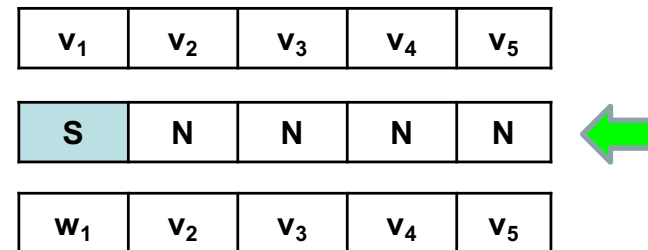
OPERADOR DE MUTACIÓN – Mutaciones para representaciones reales

- **Mutación Uniforme:** el nuevo valor del elemento que muta se elige utilizando una distribución uniforme entre el valor máximo (u_i) y mínimo (l_i) de ese parámetro. Es la mutación real más sencilla, y permite alcanzar cualquier región del espacio de búsqueda con la misma probabilidad.

Valor inicial del individuo

Etapa 1: $Muta_i$

Etapa 2: $w_i \sim U(l_i, u_i)$



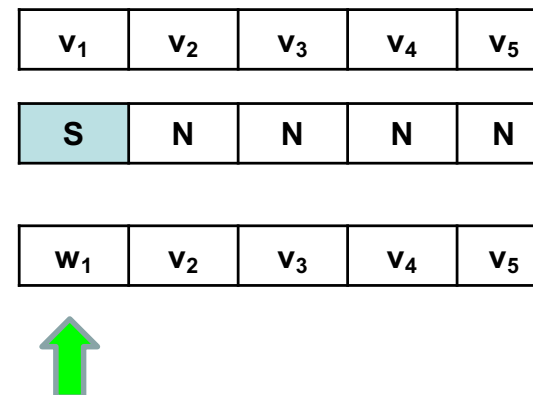
- **Mutación al Limite:** el nuevo valor del elemento que muta se elige aleatoriamente entre uno de los dos limites existentes.

Valor inicial del individuo

Etapa 1: $Muta_i$

$r_i \sim U(0,1)$

Etapa 2: $w_i = \begin{cases} l_i & \text{si } r_i < 0.5 \\ u_i & \text{c.c.} \end{cases}$



ALGORITMOS GENÉTICOS

Algoritmo 1: GA basico

x=GA (Generate, Select, Cross, Mutate, Recombine, Stop)

1 **x**=Generate ()

2 **repeat**:

3 **parents**=Select (**x**)

4 **children**=Cross (parents)

5 **children**=Mutate (children)

6 **x**=Recombine (**x**, children) 

7 **until** Stop (**x**)

8 **return** **x**

ALGORITMOS GENÉTICOS

FUNCIÓN DE RECOMBINACIÓN



- **Sustitución generacional:** Es el método más sencillo, ya que lo único que hay que hacer es **sustituir la población actual por la nueva**. Para conseguir que el tamaño de la población permanezca constante, **es necesario crear tantos hijos como individuos existen en la población**. Por otra parte, como todos los individuos existentes desaparecen, esta forma de proceder **puede hacer que se pierda la mejor solución encontrada hasta el momento**. Para evitarlo, la mejor solución obtenida a lo largo de todas las generaciones puede ser almacenada en un archivo auxiliar.
- **Elitismo:** Con el objeto de asegurar que la población mantiene la mejor solución obtenida por el GA a lo largo de todas las generaciones, se puede forzar que **la mejor solución (o el conjunto de soluciones mejores perteneciente al frente Pareto) de la población actual, pase a formar parte de la población en la próxima generación**. El resto de los individuos que la constituyen pueden ser elegidos entre los nuevos, o entre los nuevos y los antiguos. Esta forma de proceder, **denominada elitista porque asegura la supervivencia de los mejores**, puede hacer que el tamaño de la población crezca si el número de soluciones que forman el frente Pareto crece a lo largo de las iteraciones. En algunos algoritmos mutliobjetivo se limita el tamaño de la población teniendo en cuenta la dispersión de los elementos del frente.

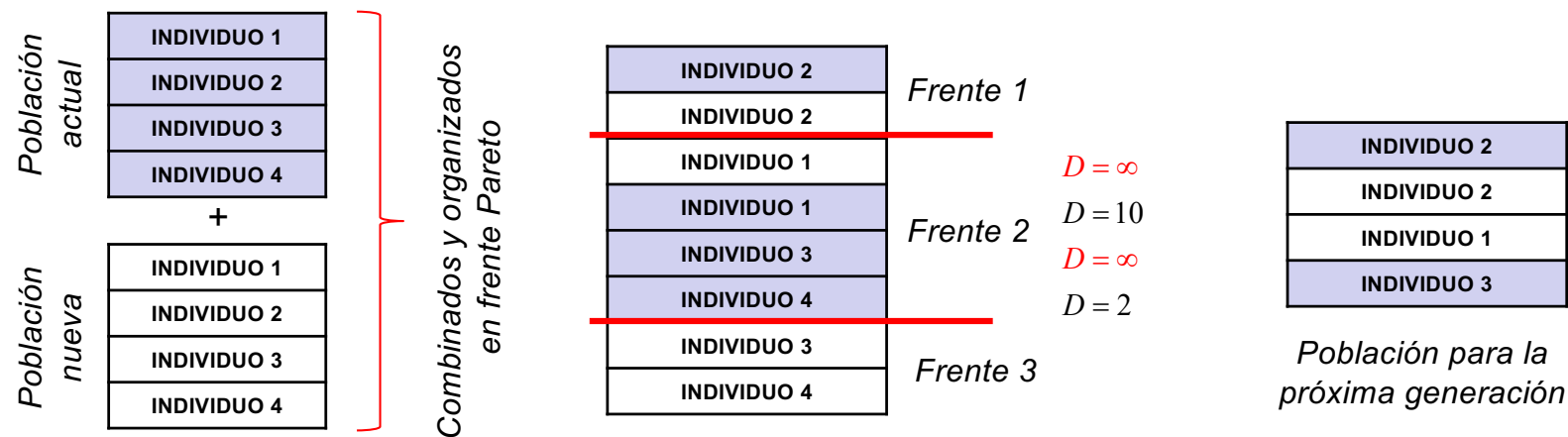


ALGORITMOS GENÉTICOS

FUNCIÓN DE RECOMBINACIÓN – NSGA-II

El método de recombinación del NSGA-II es uno de los GAs considerados estándar para problemas multiobjetivo. Para que la población de la generación siguiente tenga el mismo tamaño que la población (N_p) de la generación actual:

1. Se crea una población formada por todos los elementos de la población actual y los nuevos.
2. Se utiliza la definición Pareto para clasificar los individuos de la población total en frentes sucesivos: en el primer frente se encuentran todos los individuos que no están dominados por otros, en el segundo todos los que únicamente están dominados por individuos del primer frente, en el tercero los que están dominados por los del primero y el segundo ... y así sucesivamente.
3. Se eligen los N_p primeros elementos, para determinar el frente Pareto en el que aparece el último de ellos. Todos los individuos de los frentes anteriores forman parte de la población de la nueva generación. Para seleccionar los individuos del frente determinado que forman parte de la población en la próxima generación, se calcula la “crowding distance” de todos los individuos que constituyen el frente, y se seleccionan aquellos con mayor crowding distance.

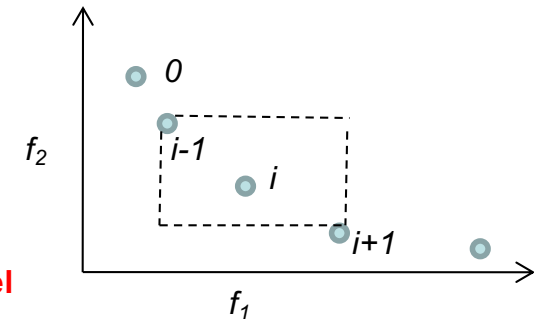


ALGORITMOS GENÉTICOS

FUNCIÓN DE RECOMBINACIÓN – NSGA-II

La “crowding distance” de un conjunto de individuos (en el caso del NSGA-II, aquellos que se encuentran dentro del frente Pareto que queremos dividir) se obtiene:

1. Calculando los valores de las funciones de mérito de todos los individuos.
2. Para cada función de mérito, **se organizan de menor a mayor sus valores**, y se calcula la **distancia asociada a cada individuo y función de mérito** como la **separación entre el valor inmediatamente superior e inferior dividido entre la diferencia entre el valor máximo y mínimo. Para los valores de los extremos, el de la distancia asociada a dicha función de mérito es infinito.**
3. Una vez calculadas las componentes asociadas a todas las funciones de mérito e individuo, **se calcula la crowding distance sumando, para cada individuo, las componentes asociadas a todas las funciones de mérito.**



Los elementos con *crowding distance* igual a infinito resultan ser valores extremos del frente elegido, por lo que NSGA-II los elige como representantes extremos del frente. Del resto, aquellos que tienen valores mayores representan puntos en los que la dispersión es mayor, motivo por el que NSGA-II los prefiere a aquellos que tienen una distancia menor, para cubrir el frente con un número limitado de puntos.



NUBES DE PARTÍCULAS

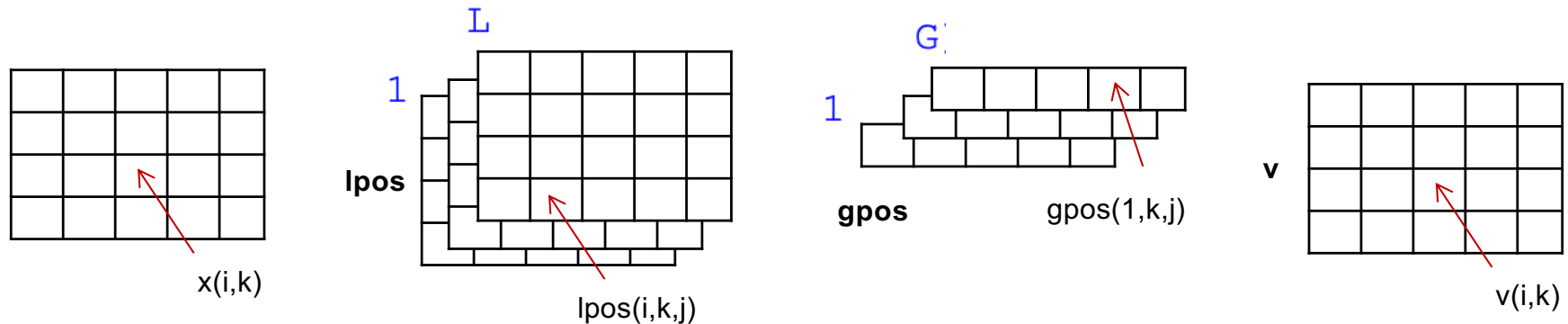
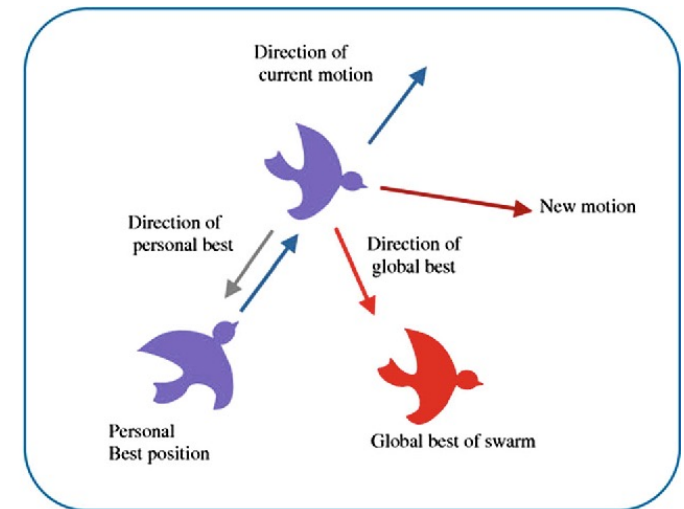
Algoritmo 2: PSO básico

gbest=PSO()

```

1  [x,v]=Initialize()
2  [lbest,gbest]=Best(x)
3  repeat:
4      [x,v]=Dynamic(x,v,lbest,gbest)
5      [lbest,gbest]=Best(x,lbest,gbest)
6  until Stop(gbest)
7  return gbest

```

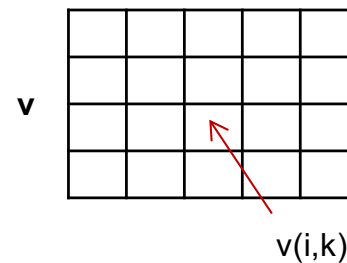
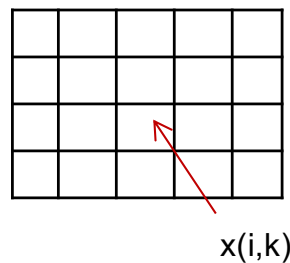


NUBES DE PARTÍCULAS

Algoritmo 2: PSO básico

gbest=PSO()

```
1 [x,v]=Initialize() ←  
2 [lbest,gbest]=Best(x)  
3 repeat:  
4     [x,v]=Dynamic(x,v,lbest,gbest)  
5     [lbest,gbest]=Best(x,lbest,gbest)  
6 until Stop(gbest) ←  
7 return gbest
```



NUBES DE PARTÍCULAS

Algoritmo 2: PSO básico

gbest=PSO()

1 **[x,v]**=Initialize()

2 **[lbest,gbest]**=Best(x) ←

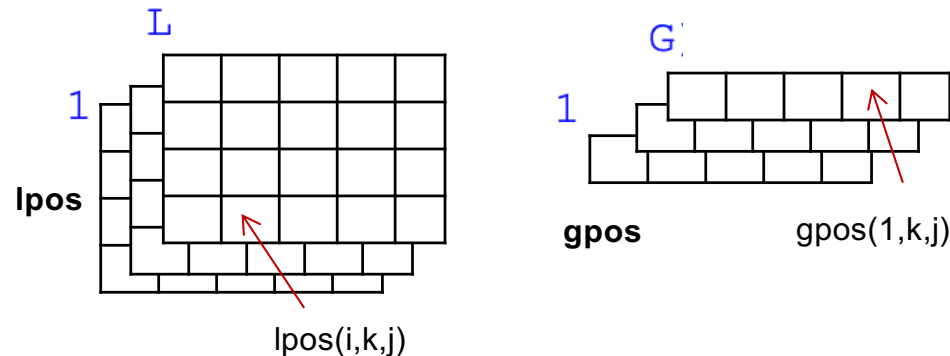
3 **repeat**:

4 **[x,v]**=Dynamic(x,v,lbest,gbest)

5 **[lbest,gbest]**=Best(x,lbest,gbest) ←

6 **until** Stop(gbest)

7 **return** gbest



NUBES DE PARTÍCULAS

Algoritmo 2: PSO básico

gbest=PSO ()

1 **[x,v]**=Initialize ()

2 **[lbest,gbest]**=Best (x)

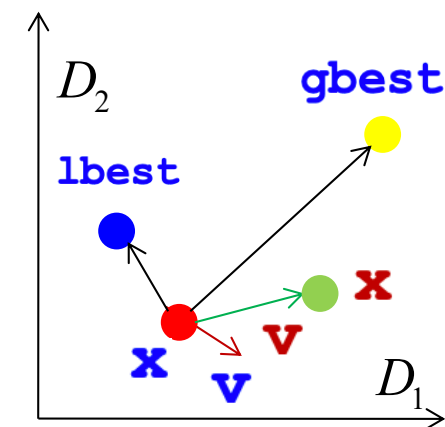
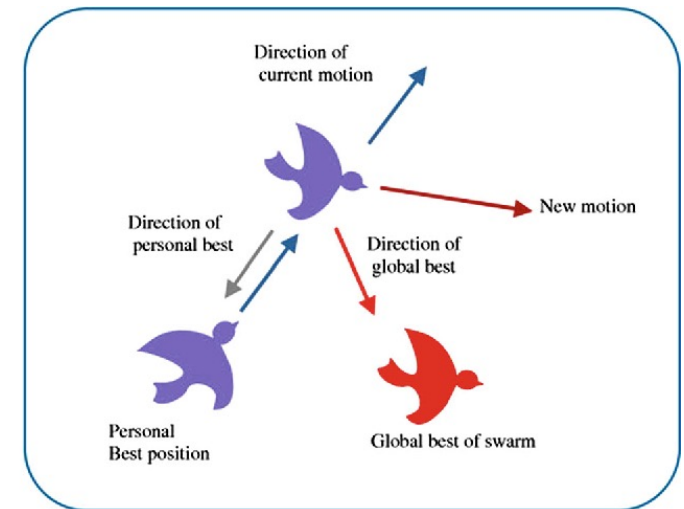
3 **repeat**:

4 **[x,v]**=Dynamic (x,v,lbest,gbest) ←

5 **[lbest,gbest]**=Best (x,lbest,gbest)

6 **until** Stop (gbest)

7 **return** gbest



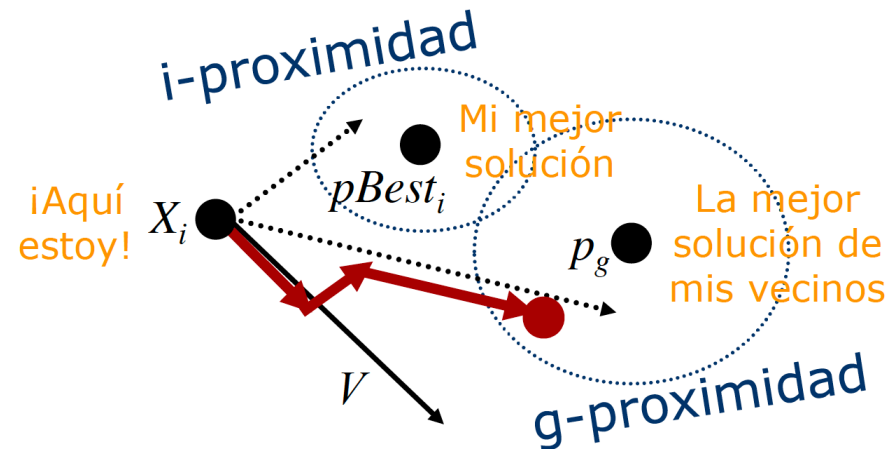
NUBES DE PARTÍCULAS

Función 1: Dinámica básica

```
[x,v]=DynamicBasic(x,v,lbest,gbest)
```

```
1 for i=1:N
2     for k=1:D
3         u_w=rand(0,1), u_l=rand(0,1), u_g=rand(0,1), s_l=rand(1,L), s_g=rand(1,G)
4         ➡ v(i,k)=w*u_w*v(i,k)+c_l*u_l*(lbest(i,k,s_l)-x(i,k))+c_g*u_g*(gbest(1,k,s_g)-x(i,k))
5         ➡ x(i,k)=x(i,k)+v(i,k)
6     end
7 end
8 return x,v
```

- $c_l = 2$ y $c_g = 2$.
- w entre 0.3 y 0.5.

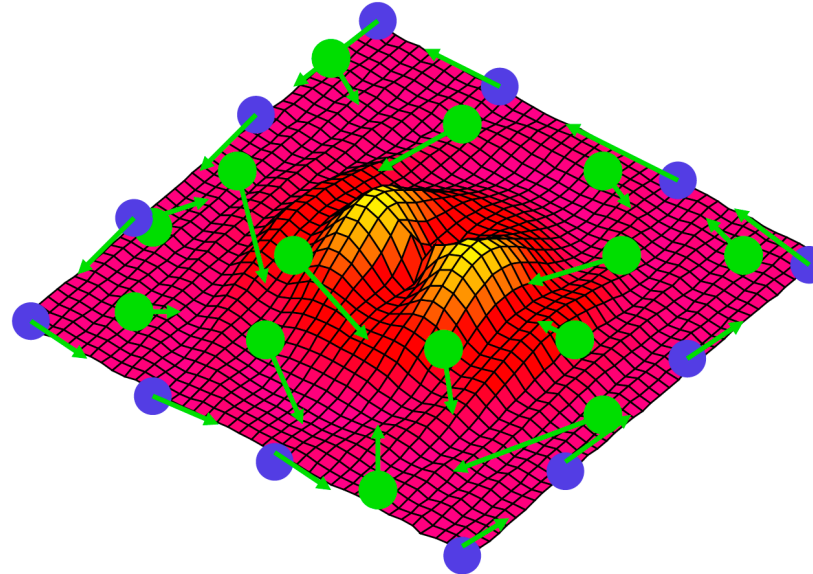


NUBES DE PARTÍCULAS

Función 2: Dinámica avanzada

```
[x,v]=DynamicAdvanced(x,v,lbest,gbest)
```

```
1 for i=1:N
2     for k=1:D
3         uw=rand(0,1), ul=rand(0,1), uc=rand(0,1), sl=rand(1,1), sg=rand(1,G)
4         v(i,k)=w*uw*v(i,k)+cl*ul*(lbest(i,k,sl)-x(i,k))+cg*ug*(gbest(1,k,sg)-x(i,k))
5         x(i,k)=x(i,k)+v(i,k)
6     end
7 end
8 [x,v]=LimitPos(x,v) ←
9 [v]=LimitSpeed(x,v) ←
10 return x,v
```



NUBES DE PARTÍCULAS

FUNCIÓN DINÁMICA – Límites

La función **LimitPos()** :

- **Posición al límite:** Se fija la posición $x(i,k)$ que se encuentra fuera del límite al valor

del límite más cercano. Tras fijar el valor de la posición, es conveniente modificar

también el valor de la velocidad correspondiente, habitualmente a través de uno de los dos procedimientos siguientes:

$$x(i,k) = \begin{cases} l_i & \text{si } x(i,k) < l_i \\ u_i & \text{si } x(i,k) > u_i \\ x(i,k) & \text{c.c.} \end{cases}$$


- **Velocidad nula:** se frena totalmente la partícula, con el objeto de que su próximo desplazamiento únicamente sea función de la posición de los mejores.

$$v(i,k) = \begin{cases} 0 & \text{si } x(i,k) < l_i \vee x(i,k) > u_i \\ v(i,k) & \text{c.c.} \end{cases}$$

- **Velocidad opuesta:** se cambia el signo de la velocidad de la partícula, de forma que esta se comporte como si su velocidad hubiese sido reflejada por las paredes de los límites de las variables de decisión.

$$v(i,k) = \begin{cases} -v(i,k) & \text{si } x(i,k) < l_i \vee x(i,k) > u_i \\ v(i,k) & \text{c.c.} \end{cases}$$


- **Choque elástico:** la nueva posición y velocidad de la partícula se determina teniendo en cuenta su posición y velocidad de forma que **su comportamiento simule el que se da cuando se produce un choque elástico sobre las paredes** definidas por los límites.

$$x(i,k) = \begin{cases} l_i - v(i,k) & \text{si } x(i,k) < l_i \\ u_i - v(i,k) & \text{si } x(i,k) > u_i \\ x(i,k) & \text{c.c.} \end{cases}$$

$$v(i,k) = \begin{cases} -v(i,k) & \text{si } x(i,k) < l_i \vee x(i,k) > u_i \\ v(i,k) & \text{c.c.} \end{cases}$$


NUBES DE PARTÍCULAS

FUNCIÓN DINÁMICA – Límites

La función **LimitSpeed()** :

- **Anular**, para que **se frenen totalmente**

$$v(i,k) = \begin{cases} 0 & \text{si } |v(i,k)| > \frac{u_i - l_i}{2} \\ v(i,k) & \text{c.c.} \end{cases}$$



- **Limitar**, para que **alcancen el valor del límite impuesto**

$$v(i,k) = \begin{cases} \frac{u_i - l_i}{2} & \text{si } v(i,k) > \frac{u_i - l_i}{2} \\ -\frac{u_i - l_i}{2} & \text{si } v(i,k) < -\frac{u_i - l_i}{2} \\ v(i,k) & \text{c.c.} \end{cases}$$



Algoritmo 3: PSO con mutación

gbest=PSO()

1 [**x**,**v**]=Initialize()

2 [**lbest**,**gbest**]=Best(**x**)

3 **repeat**:

4 [**x**,**v**]=Dynamic(**x**,**v**,**lbest**,**gbest**)

5 **x**=Mutation(**x**) 

6 [**lbest**,**gbest**]=Best(**x**,**lbest**,**gbest**)

7 **until** Stop(**gbest**)

8 **return** **gbest**

Algoritmo 4: DE básico

x=DE ()

1 **x**=Initialize ()

2 **repeat**:

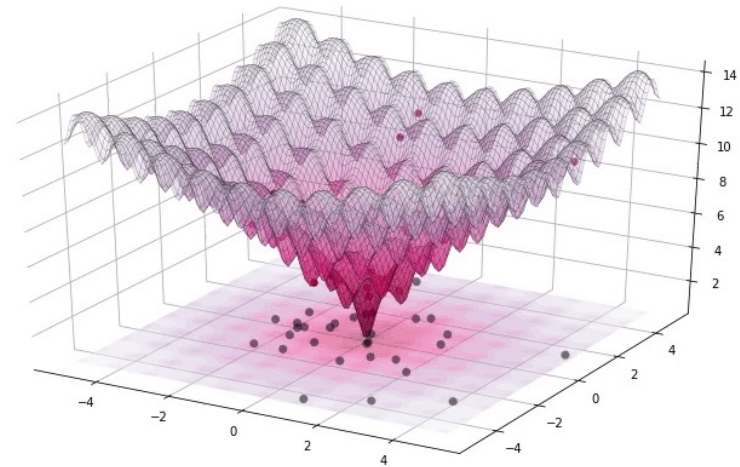
3 **x_new**=Pertubate (**x**)

4 **x_new**=Cross (**x**, **x_new**)

5 **x**=Recombine (**x**, **x_new**)

6 **until** Stop (**x**)

7 **return** **x**



EVOLUCIÓN DIFERENCIAL

Algoritmo 4: DE básico

x=DE ()

1 **x**=Initialize () 

2 **repeat**:

3 **x_new**=Pertubate (x)

4 **x_new**=Cross (x,x_new)

5 **x**=Recombine (x,x_new)

6 **until** Stop (x) 

7 **return** **x**

EVOLUCIÓN DIFERENCIAL

Algoritmo 4: DE básico

x=DE ()

1 **x**=Initialize ()

2 **repeat**:

3 **x_new**=Pertubate (x) 

4 **x_new**=Cross (x, x_new)

5 **x**=Recombine (x, x_new)

6 **until** Stop (x)

7 **return** **x**

EVOLUCIÓN DIFERENCIAL

Función 3: Perturbacion

```
x_new=Perturbate(x)
```

```
1 F=ObtainF()           %Fixed
```

```
2 for i=1:N
```



```
3   [base,d1,d2]=SelectIndex(x)
```

```
4   F=ObtainF()         %Dither
```

```
5   for k=1:D
```

```
6     F=ObtainF(); %Jitter
```

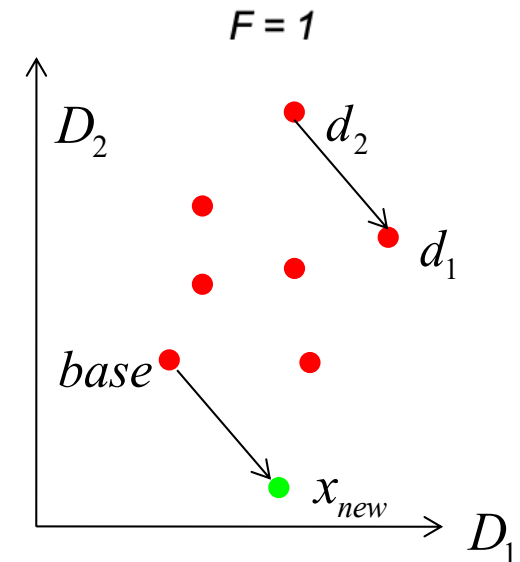


```
7     x_new(i,k)=x(base,k) + F(x(d1,k) - x(d2,k))
```

```
8   end
```

```
9 end
```

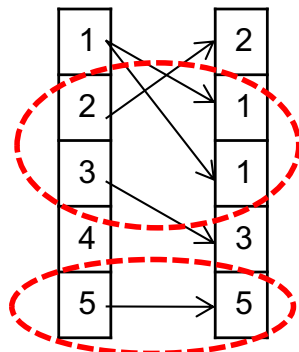
```
10 return x_new
```



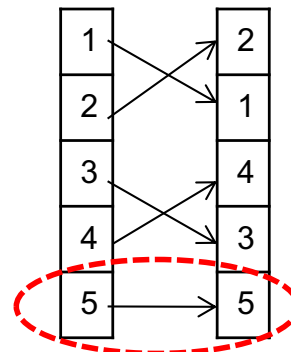
EVOLUCIÓN DIFERENCIAL

FUNCIÓN DE PERTURBACIÓN – Selección de la base

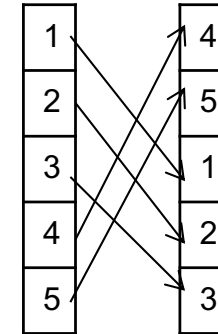
- **Selección entre los mejores**: una opción consiste en **elegir como la base el mejor elemento**, o en los casos en los que haya un conjunto de individuos igualmente buenos, un individuo de dicho conjunto.
- **Selección aleatoria**: el valor de los índices **se elige de forma aleatoria** entre los N posibles sin tener en cuenta la aptitud (i.e. los valores de las funciones de mérito) sobre la población de soluciones del problema.
 - **Uniforme**: se utiliza una distribución uniforme entre 1 y N para **elegir, con la misma probabilidad, cualquier individuo de la población**.
 - **Basada en perturbación**: para asegurar que **cada solución es elegida una única vez**.
 - **Basada en un offset**: para asegurar que **cada solución es elegida una única vez, y que el índice de la solución de la base no coincide con el índice i de la nueva solución**.



Uniforme



Perturbación



Offset

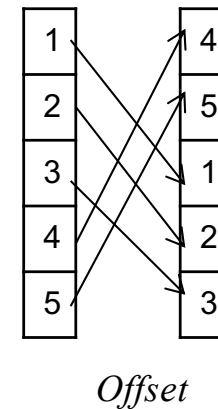
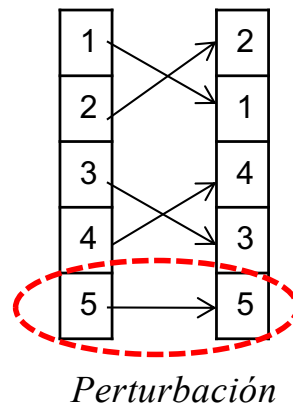
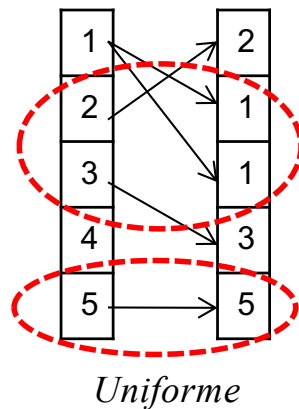
EVOLUCIÓN DIFERENCIAL

FUNCIÓN DE PERTURBACIÓN – Selección de los individuos de la perturbación

La elección de los elementos utilizados para realizar la perturbación, y por lo tanto el **valor de los índices $d1$ y $d2$** , se hace siempre **de forma aleatoria entre los índices existentes**.

Además, **para evitar que la perturbación sea nula (caso que se da cuando $d1$ y $d2$ son iguales), o la coincidencia de cualquiera de los dos índices con el valor del índice de la base o el del nuevo individuo i** , habitualmente:

- Se elige el valor de $d1$ una vez que ya se conoce el de la base, utilizando una distribución uniforme sobre todos los índices salvo el de la base e i .
- A continuación, se elige el valor de $d2$, utilizando una distribución uniforme sobre todos los índices salvo el de la base, i y $d1$.



EVOLUCIÓN DIFERENCIAL

FUNCIÓN DE PERTURBACIÓN – Elección de los valores de F

- Localización del cálculo de la F

- Fuera del bucle principal, con el objeto de usar **la misma F en todos los individuos de la población en cada iteración**.
- **Para cada individuo de la población**, antes del bucle que itera sobre cada una de las variables de decisión.
- **Para cada individuo y variable de decisión**, dentro de los bucles.

- Obtención del valor de la F

- Utilizar un valor prefijado por el usuario. Una variante consiste en prefijar un valor para cada variable de decisión.
- Generar un número aleatoriamente utilizando una **distribución uniforme entre un valor máximo y mínimo**.
- Generar un numero aleatorio utilizando una **distribución Gaussiana con una media y varianza dadas**.

Habitualmente, si se procede de esta forma, **el valor medio se fija a cero, y la varianza se iguala a 1**.

Los valores de F del primer caso, o sus límites en el segundo, se eligen dentro del rango $[0,1]$. El valor inferior de 0 a veces se sustituye por el valor F_{crit} , donde p_{cr} es el valor de la probabilidad de cruce asociado a una función $Cross()$ del tipo uniforme (su comportamiento se especifica en la siguiente sección).

$$F_{crit} = \sqrt{\frac{1 - p_{cr}}{N}}$$

EVOLUCIÓN DIFERENCIAL

Algoritmo 4: DE básico

x=DE ()

1 **x**=Initialize ()

2 **repeat**:

3 **x_new**=Pertubate (x)

4 **x_new**=Cross (x, x_new)

5 **x**=Recombine (x, x_new)

6 **until** Stop (x)

7 **return** **x**

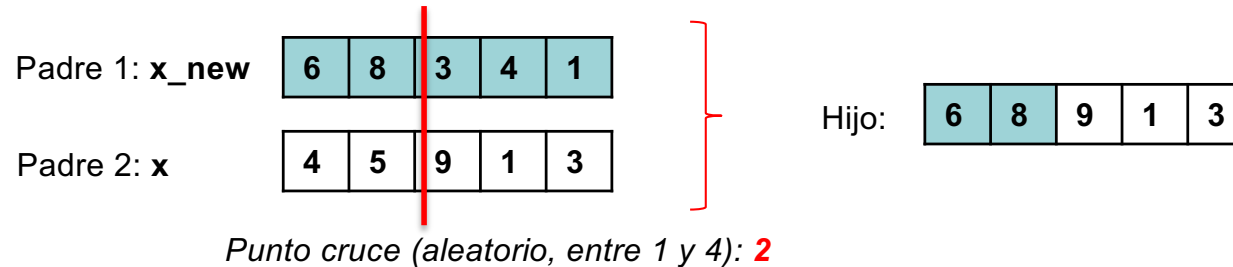


EVOLUCIÓN DIFERENCIAL

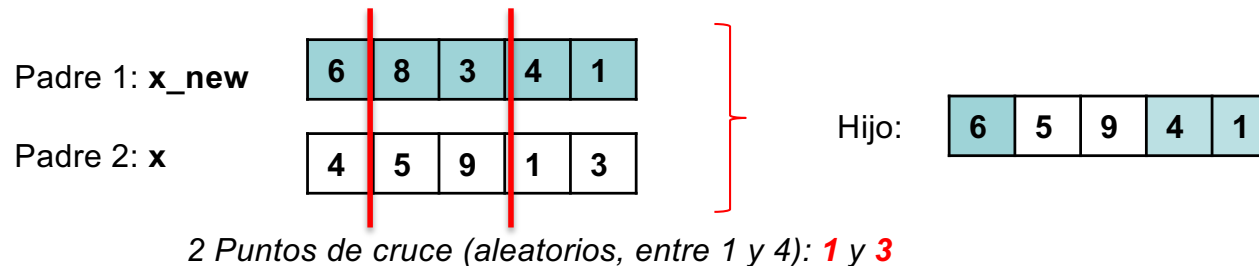
FUNCIÓN DE CRUCE

La función de cruce combina la información de los individuos de la población existente $\mathbf{x}$ con los individuos $\mathbf{x}_{\text{new}}$ obtenidos en la operación de perturbación. **La combinación se hace por parejas, siguiendo el orden establecido en las dos poblaciones (i.e., el individuo i -ésimo en $\mathbf{x}$ con el individuo i -ésimo en $\mathbf{x}_{\text{new}}$).**

- **Cruce por 1 punto:** se determina de forma aleatoria, siguiendo una distribución habitualmente uniforme, el punto de cruce de las dos cadenas, y los hijos se crean concatenando la cadena de un padre con la del otro.

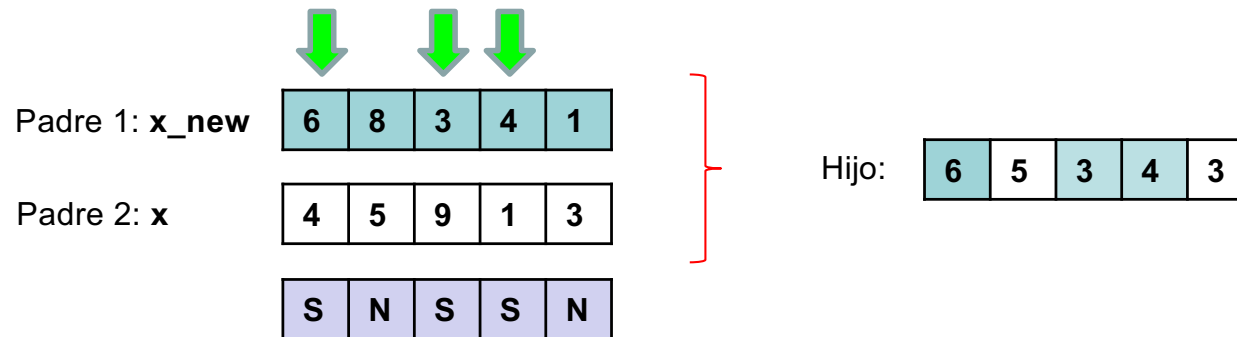


- **Cruce por N puntos:** se determinan de forma aleatoria, siguiendo una distribución habitualmente uniforme, los puntos de cruce de las dos cadenas, y los hijos se crean concatenando, de forma alternada, las cadenas de un padre con las del otro.



EVOLUCIÓN DIFERENCIAL

- **Cruce uniforme**: se determina de forma aleatoria, a partir de una probabilidad p_{cr} , si cada elemento de la cadena del primer padre pertenece al primer hijo o no.



Aleatoriamente se determina que los elementos 1, 3 y 4 se toman del mismo padre, y los elementos 2 y 5 del contrario.

➡ De forma general, se considera que un valor de $p_{cr} < 0.2$ es adecuado, como punto de partida, para DE.

EVOLUCIÓN DIFERENCIAL

Algoritmo 4: DE básico

x=DE ()

1 **x**=Initialize ()

2 **repeat**:

3 **x_new**=Pertubate (x)

4 **x_new**=Cross (x, x_new)

5 **x**=Recombine (x, x_new) ←

6 **until** Stop (x)

7 return **x**

EVOLUCIÓN DIFERENCIAL

FUNCIÓN DE RECOMBINACIÓN

La función de recombinación elige, entre los individuos de la población existente $\mathbf{x}$ y los nuevos $\mathbf{x_new}$ obtenidos a partir de las operaciones de perturbación y mutación, los individuos que formarán parte de la población en la próxima iteración del algoritmo.

La selección se hace por parejas, siguiendo el orden establecido en las dos poblaciones. Es decir, el i -ésimo individuo de la población resultante se elige entre el i -ésimo individuo de $\mathbf{x}$, y el i -ésimo individuo de $\mathbf{x_new}$.



En esta **comparación por pares ordenados**:



- **Se elige el mejor de los dos**, teniendo en cuenta las funciones de mérito del problema que se desea resolver y la función de optimalidad elegida.
- **En el caso en el que los dos sean equivalentes, se elige el individuo de la nueva población.**

Esta forma de proceder es muy sencilla, y puede ser aplicada sin ninguna dificultad a problemas monoobjetivo, multiobjetivo, o problemas con restricciones, ya que únicamente es necesario determinar cual es el mejor de cada pareja.

El método de proceder se puede considerar **elitista**, ya **que conserva al menos uno de los mejores individuos encontrados hasta el momento**. Sin embargo, es importante tener en cuenta que **se pueden perder mejoras intermedias**, cuando el nuevo individuo es mejor que buena parte de los individuos de la población existente, pero peor que su pareja correspondiente. **Este hecho, no tiene porque ser necesariamente negativo, ya que puede reducir la presión de búsqueda del algoritmo sobre zonas de bondad intermedia.**



COMPARATIVA

1. Los mecanismos utilizados para crear las nuevas soluciones:



- Los **GAs** crean las nuevas soluciones por medio de dos operaciones que cruzan (combinan) y mutan (modifican) parte de la información proveniente de las soluciones existentes en la población anterior, seleccionados en base a su aptitud. Es decir, **las nuevas soluciones combinan la información asociada habitualmente a buenas soluciones con otra introducida, de forma puramente aleatoria, por el proceso de mutación.**
- Los **PSOs** crean nuevas soluciones desplazando las existentes hacia, 1) las mejores soluciones obtenidas por toda la población, 2) la mejor que cada individuo ha encontrado, 3) el sentido y dirección seguido en la etapa anterior. Por lo tanto, **combinan las soluciones actuales, buenas o malas, con la mejor historia pasada (global y local).**
- Los **DEs** crean las nuevas soluciones: 1) perturbando habitualmente todas las soluciones existentes teniendo en cuenta la distancia entre dos soluciones existentes cualquiera, y 2) combinando el valor obtenido con una de las existentes. Es decir, **modifican las soluciones existentes teniendo en cuenta la dispersión e información contenida en la población actual, sin tener en cuenta en las formulaciones más básicas la aptitud de las soluciones.**

COMPARATIVA

2. Los mecanismos utilizados para decidir que soluciones forman parte de la nueva generación:



- Aunque los **GAs** permiten el uso de diferentes mecanismos para llevar a cabo esta etapa de decisión, los que predominan son las **técnicas de sustitución elitistas**, en las que todos los individuos de la población actual y todos los individuos nuevos compiten entre si, con el objeto de **asegurar que las mejores soluciones creadas en todo momento son las que sobreviven**.
- Los **PSOs** mantienen por separado varios conjuntos de soluciones: 1) la nube en la que todas las soluciones nuevas son utilizadas como punto de partida de la próxima iteración del algoritmo, 2) el conjunto de los mejores globales en los que las mejores soluciones alcanzadas por todas las partículas son almacenadas, y 3) el conjunto de los mejores locales de cada partícula donde se almacena la mejor solución obtenida por cada solución de la nube durante todo el proceso. Es decir, esta familia de algoritmos **separa las soluciones sobre las que trabaja (nube) de la historia pasada (mejores globales y locales)**.
- Los **DEs** comparan cada solución recientemente creada con una solución existente en la población anterior, haciendo que sobreviva la mejor de cada comparación. Las soluciones mejores a las de una parte del conjunto de la población se pierden si éstas son enfrentadas contra una mejor que ellas, por lo que **el mecanismo elegido no asegura la supervivencia de las mejores del conjunto, si no de una muestra de las mismas**.

Por lo tanto, por una parte **los GAs elitistas pueden converger más rápido a una zona del espacio**, al eliminar rápidamente soluciones que colectivamente no son muy buenas. Por la otra, **los PSOs y DEs convergen más lentamente**, al no tomar como punto de partida los mejores de todos los existentes.

COMPARATIVA

3. El tipo de codificación sobre la que trabajan:



- Los **GAs**, que fueron originariamente desarrollados sobre codificaciones binarias, utilizan en la actualidad **codificaciones binarias, enteras y reales**.
- Aunque los **PSOs** fueron originariamente desarrollados sobre **codificaciones reales**, ya existen algunas **versiones especiales para codificaciones enteras**.
- Los **DEs** trabajan únicamente sobre **codificación reales**.

Por lo tanto, desde el punto de vista de las codificaciones, **los GAs son en este momento la herramienta más versátil** de las tres vistas a lo largo de este tema. Teniendo en cuenta que los DEs y PSOs son técnicas con bastantes menos años de desarrollo, es posible que vayan apareciendo nuevas variantes que incorporen nuevas codificaciones.

4. Complejidad Computacional:



- Los **PSOs y DEs son habitualmente técnicas computacionalmente más sencillas que los GAs**, y por lo tanto, **sus versiones más básicas suelen ser mas fáciles de implementar para construir un prototipo rápido** con el que intentar resolver el problema de optimización deseado.
- **Las versiones más elaboradas de todas ellas tienen complejidades similares**, ya que muchas nuevas versiones de PSOs y DEs incorporan funcionalidades originales de los GAs, perdiendo de esta forma su simplicidad original.

TRABAJO PRÁCTICO A ENTREGAR

La entrega correspondiente al Tema 3 consiste en la implementación de alguna de las metaheurísticas basadas en búsquedas evolutivas para resolver el problema planteado en el Tema 1. Además, en esta entrega se compararán los resultados obtenidos por la técnica utilizada en el Tema 2 con la técnica implementada en esta entrega.

La entrega se ha dividido en **dos partes**:

- **7 de Junio:** Propuesta con el planteamiento del problema (representación de las soluciones, operadores, funciones objetivo, etc.), y justificación del algoritmo seleccionado, para que los profesores de la asignatura la analicen, y comenten con los alumnos.
- **21 de Junio:** Presentación de un guión, en el que se recoja la implementación final del algoritmo, los resultados obtenidos y una discusión sobre los mismos. En este guión será necesario incluir:
 - Una descripción del funcionamiento de los **operadores** seleccionados.
 - Una explicación del **método utilizado para evaluar las soluciones** (coste + tratamiento de las restricciones).
 - Los valores a los que se han fijado los **parámetros del algoritmo** escogido.
 - Una **discusión de los resultados obtenidos**, en la que se debe poner de manifiesto:
 - si se **mejoran los puntos de partida del algoritmo** y si **se cumplen las restricciones**.
 - si el algoritmo evolutivo **mejora las soluciones obtenidas con el algoritmo de búsqueda local** utilizado en el tema anterior.